Lecture 5: The Complexity Class **NP**

Dr. George Mertzios

george.mertzios@durham.ac.uk

Room 2066, MCS Building
Tel: 42429

Beyond **P**

- We have seen that the class **P** provides a useful model for easy problems.
- This includes 2-SATISFIABILITY and 2-COLOURABILITY.
- But what about 3-SATISFIABILITY and 3-COLOURABILITY?
- No polynomial time algorithms for these problems are known.
- On the other hand...

Certificates

Every yes-instance of such problems has a short and easily checkable certificate.

- SATISFIABILITY — a satisfying assignment
- k-COLOURABILITY — a proper colouring
- HAMILTONIAN CYCLE — a Hamiltonian cycle
- TRAVELLING SALESPERSON — a permutation (i.e., a route)

Verifiers

Definition

An acceptor machine V which halts on all inputs is called a *verifier* for a language $\mathcal{L}$ if

$$\mathcal{L} = \{w \mid V \text{ accepts “}w; c\text{” for some string } c\}$$

- The string c is called a certificate (or witness) for w.
- A verifier is said to be polynomial-time if it is a polynomial-time TM, and there is a polynomial $p(x)$ such that, for any $w \in \mathcal{L}$, there is a certificate c with $|c| \leq p(|w|)$.
- All problems from the previous slide have polynomial-time verifiers

The class **NP**

Definition

The class of languages that have polynomial-time verifiers is called **NP**.

The class **NP**

Definition

The class of languages that have polynomial-time verifiers is called **NP**.

- More Examples of Problems in **NP**:

COMPOSITE NUMBER
Instance: A positive integer k
Question: Are there integers $u, v > 1$ such that $u \cdot v = k$?

SUBSET SUM
Instance: A collection of positive integers $S = \{a_1, \ldots, a_k\}$ and a target integer t
Question: Is there a subset $T \subseteq S$ such that $\sum_{i \in T} a_i = t$?

Problems (probably) not in **NP**

NO HAMILTONIAN CYCLE
Instance: A graph G
Question: Is it true that G has no Hamiltonian cycle?

Question: How to provide verifiable evidence that a graph does not have a Hamiltonian Cycle?

Problems (probably) not in **NP**

NO HAMILTONIAN CYCLE
Instance: A graph G
Question: Is it true that G has no Hamiltonian cycle?

Question: How to provide verifiable evidence that a graph does not have a Hamiltonian Cycle?

CHECKERS
Instance: An integer n and a position in checkers on $n \times n$ board
Question: Is it a winning position for White?

Question: How to provide verifiable evidence that a for every sequence of the opponent's moves there is a winning strategy for White, when starting from this position?

Nondeterministic Machines

We can get an alternative definition of the class **NP** by considering non-deterministic machines.

Recall that if NT is a non-deterministic Turing Machine, then $NT(x)$ denotes the tree of configurations which can be entered with input x, and NT accepts x if there is some accepting path in $NT(x)$.

Definition

The time complexity of a non-deterministic Turing Machine NT is the function NTime_{NT} such that $\text{NTime}_{NT}(x)$ is the number of steps in the shortest accepting path of $NT(x)$ if there is one, otherwise it is the number of steps in the shortest rejecting path.

(If not all paths of $NT(x)$ halt, then $\text{NTime}_{NT}(x)$ is undefined.)

Non-Deterministic Time Complexity

Definition

For any function f, we say that the non-deterministic time complexity of a decidable language $\mathcal{L}$ is $O(f)$ if there exists a non-deterministic TM NT which decides $\mathcal{L}$, and constants n_0 and c such that for all inputs x with $|x| > n_0$,

$$NTime_{NT}(x) \leq c \cdot f(|x|)$$

Definition

The non-deterministic time complexity class NTIME[f] is defined to be the class of all problems (i.e. languages), for which there exists an algorithm with non-deterministic time complexity in $O(f)$.

An Alternative Definition of **NP**

Definition

$$\mathbf{NP} = \bigcup_{k \geq 0} NTIME[n^k]$$

This was the original form of the definition of **NP**, and was first formulated by Richard Karp in 1972.

It explains the name — **NP**= Nondeterministic Polynomial-time

Equivalence

Theorem

The two different definitions of **NP** *are equivalent*

Proof :

- If $\mathcal{L} \in NTIME[n^k]$, then there is a non-deterministic machine NT such that $x \in \mathcal{L}$ if and only if there is an accepting computation path in $NT(x)$. Furthermore, the length of these paths is $O(|x|^k)$.

Equivalence

Theorem

The two different definitions of **NP** *are equivalent*

Proof :

- If $\mathcal{L} \in NTIME[n^k]$, then there is a non-deterministic machine NT such that $x \in \mathcal{L}$ if and only if there is an accepting computation path in $NT(x)$. Furthermore, the length of these paths is $O(|x|^k)$.
- Using (some encoding of) these computation paths as the certificates, we can construct a polynomial time verifier for $\mathcal{L}$ which simply checks that each step of the computation path is valid.

Equivalence proof (continued)

Theorem

The two different definitions of **NP** *are equivalent*

- Conversely, if $\mathcal{L}$ has a polynomial-time verifier V, then we can construct a non-deterministic machine that first "guesses" the value of the certificate (by making a series of non-deterministic choices), and then simulates V with that certificate.

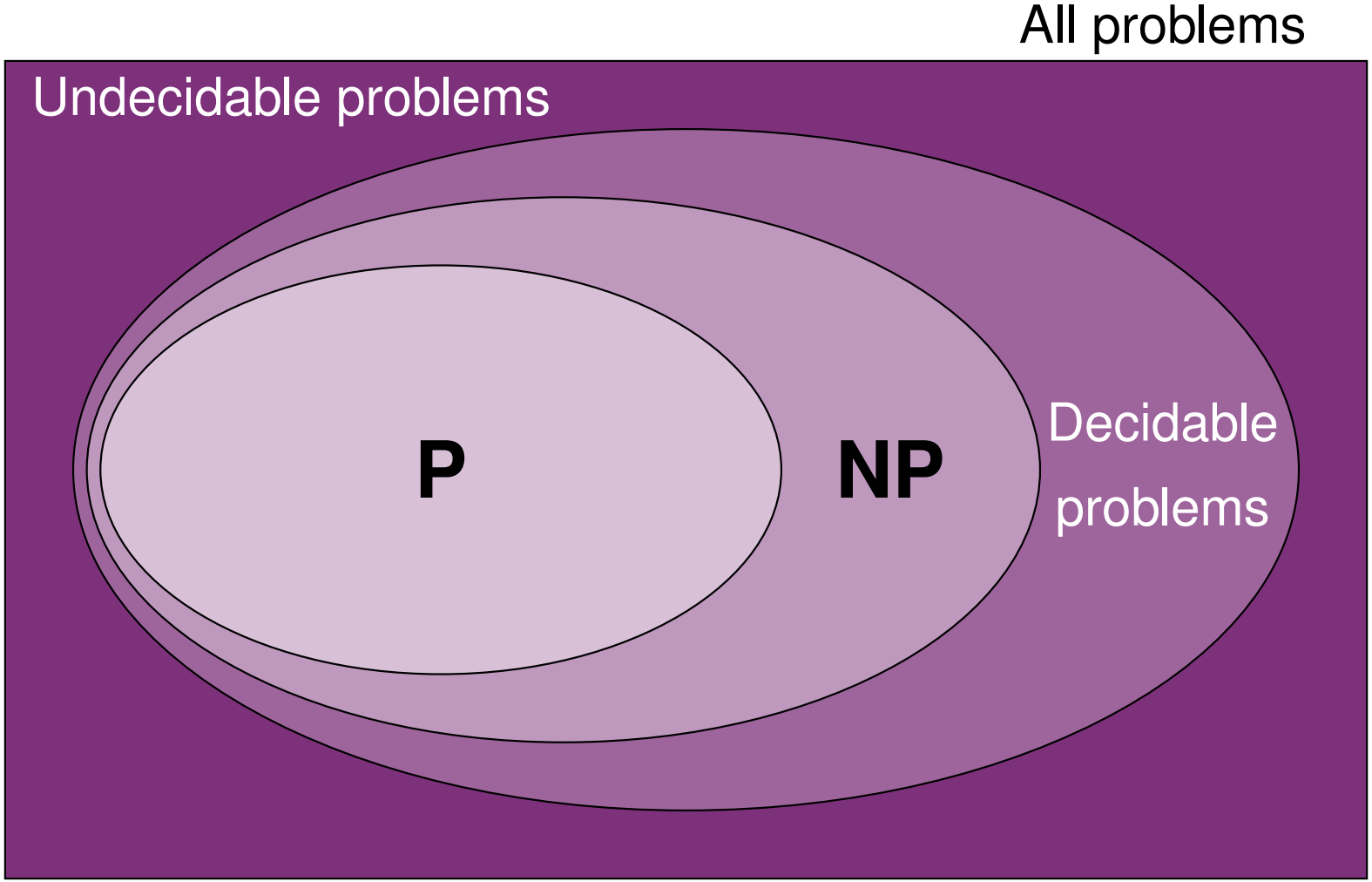
Equivalence proof (continued)

Theorem

The two different definitions of **NP** *are equivalent*

- Conversely, if $\mathcal{L}$ has a polynomial-time verifier V, then we can construct a non-deterministic machine that first "guesses" the value of the certificate (by making a series of non-deterministic choices), and then simulates V with that certificate.
- Since the length of the certificate is polynomial in the length of the input, this machine is a non-deterministic polynomial-time decision procedure for $\mathcal{L}$.

P and **NP**

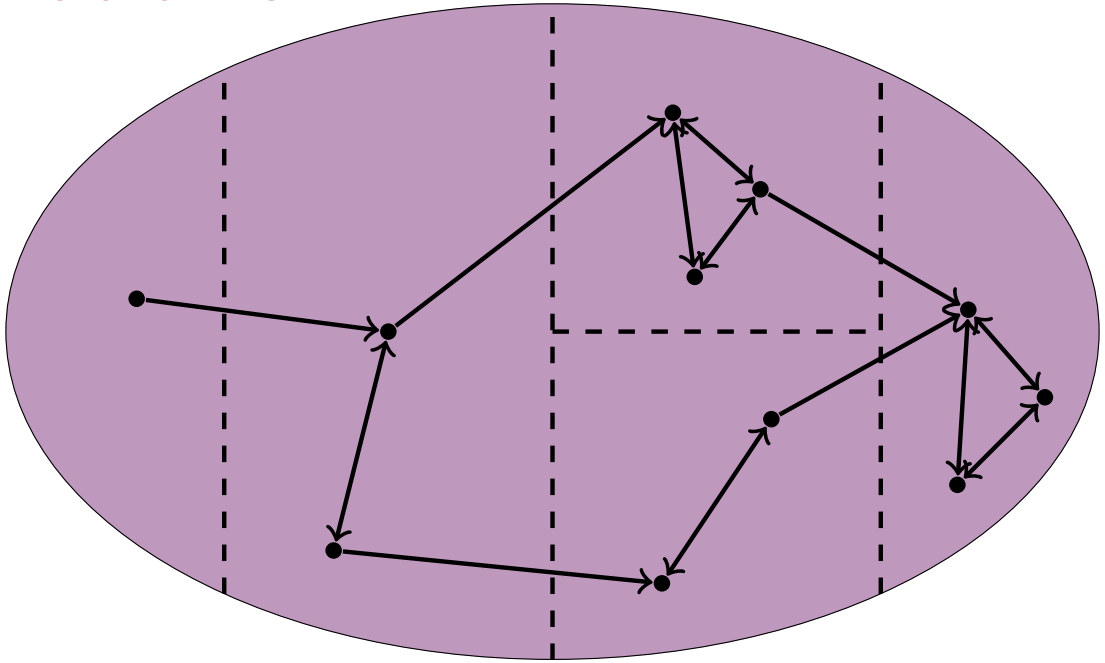

All problems
Undecidable problems
P
NP
Decidable
problems

$\mathbf{P} = \mathbf{NP}$?

The question of whether or not $\mathbf{P} = \mathbf{NP}$ is one of the biggest open problems in computer science.

- It is equivalent to determining whether or not the existence of a short solution guarantees an efficient way to find that solution.
- Virtually everyone is convinced that $\mathbf{P} \neq \mathbf{NP}$, but after more than 40 years of effort there is still no proof.
- Resolving this question (either way) would win $1M, see http://www.claymath.org/millennium-problems/P-vs-NP-problem
- For 116 claimed solutions to this question, see https://www.win.tue.nl/g̃woegi/P-versus-NP.htm

Complete Problems



- Any complexity class can be partitioned into equivalence classes via polynomial-time reduction — each class contains problems that are reducible to each other (i.e. “equivalent”).
- These equivalence classes are partially ordered by reduction.
- Problems in the maximal class are called complete (there is –by definition– no problem in the class, which is strictly harder).

NP-completeness

- **NP**-complete problems are the hardest problems in **NP**.
- They are all equally difficult (up to some polynomial factor): an efficient solution to one would solve them all at once!
- To show that $\mathcal{L}$ is **NP**-complete we must show that every language in **NP** can be reduced to $\mathcal{L}$ in polynomial time.
- This would appear to be hard!
- However, once we have one NP-complete language $\mathcal{L}_0$, we can show any other language $\mathcal{L}$ is NP-complete "just" by showing that $\mathcal{L}_0 \leq \mathcal{L}$
- Hence the most difficult part is finding the first one...

The First **NP**-complete Problem

This major first step was done independently by Steven Cook and Leonid Levin around 1970-71.

Theorem (Cook-Levin)

SATISFIABILITY *is* **NP***-complete.*

Corollary

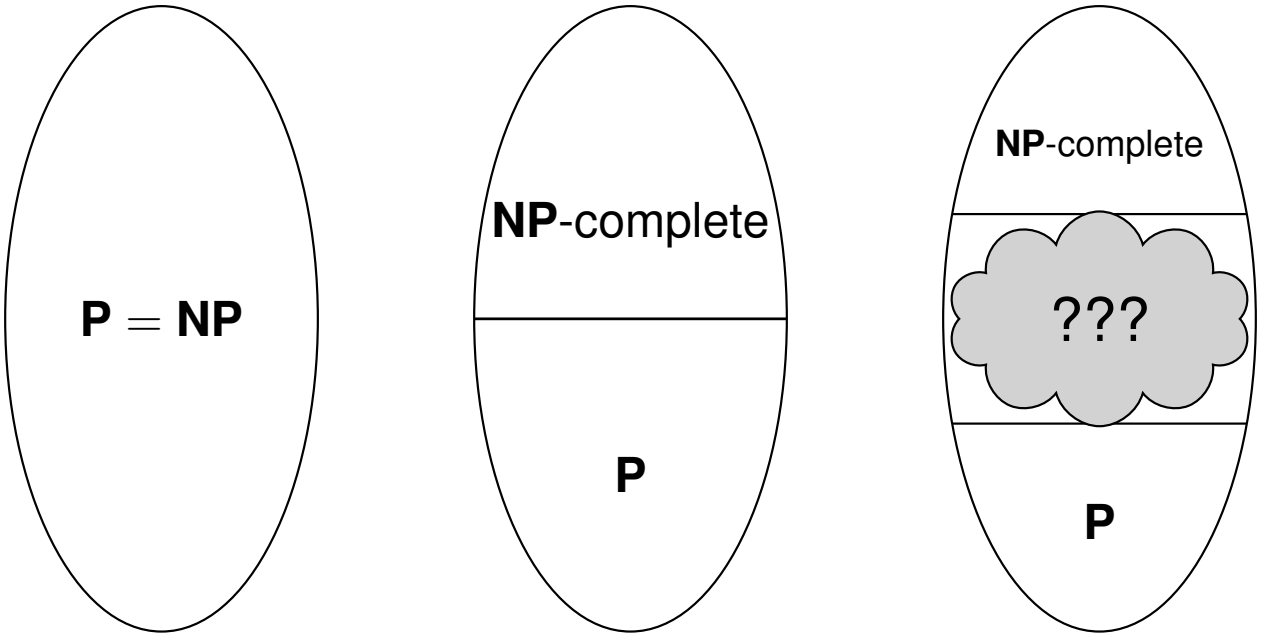
P $=$ **NP** *if and only if* SATISFIABILITY *is in* **P***.*

A very short proof that $\mathbf{P} \neq \mathbf{NP}$

- Proof by contradiction.
- Assume $\mathbf{P} = \mathbf{NP}$. Let y be a proof that $\mathbf{P} = \mathbf{NP}$.
- Then, y can be efficiently verified by a competent computer scientist, the existence of which we assert.
- However, since $\mathbf{P} = \mathbf{NP}$, the proof y can be efficiently generated by such computer scientists.
- This generation has not yet occurred (despite attempts by such computer scientists), we have a contradiction.
- Shall we claim $1M ?

Interesting questions

- Do complexity classes contain only easy and hard problems?
- Can we classify any problem in **NP** as polynomial or **NP**-complete?
- Which of these diagrams corresponds to a true picture of **NP**?



Ladner's theorem

Theorem (Ladner 1975)

If **P** $\neq$ **NP** *then* **NP** *contains infinitely many (polynomial-time) inequivalent problems.*

- This implies that unless **P** $=$ **NP**, the class **NP** contains (infinitely many) problems that are neither in **P** nor **NP**-complete. Such problems are called **NP**-intermediate.
- The proof is non-constructive (by diagonalisation).
  For details, see e.g. Bovet-Crescenzi Chapter 5.3.
- What problems are (possible candidates for) **NP**-intermediate?

Three old problems

- Obviously, a proof that some problem is **NP**-intermediate would imply that **P** $\neq$ **NP**, so we are not going to prove that any problems are **NP**-intermediate.

Three old problems

- Obviously, a proof that some problem is **NP**-intermediate would imply that **P** $\neq$ **NP**, so we are not going to prove that any problems are **NP**-intermediate.
- In their 1979 textbook, Garey and Johnson highlighted three problems in **NP** whose complexity was undecided (and so, who were possible candidates):
  - LINEAR PROGRAMMING
  - PRIMES/COMPOSITE
  - GRAPH ISOMORPHISM

  Let's look at each of these problems . . .

Linear Programming

Linear Programming (LP)
Instance: Integer vectors $V_i = (v_1^i, \ldots, v_n^i)$, $1 \leq i \leq m$, $D = (d_1, \ldots, d_n)$, $C = (c_1, \ldots, c_n)$, and an integer B
Question: Is there a rational vector $X = (x_1, \ldots, x_n)$ such that $V_i \cdot X \leq d_i$ for all $1 \leq i \leq m$ and such that $C \cdot X \geq B$?

- Essentially, this is a problem of maximizing a linear function subject to linear constraints.
- In 1979, Leonid Khachiyan proved that this problem is in **P**. LP-based algorithms are now among most popular ways to solve algorithmic problems.
- Note that a similar problem Integer Programming where X is required to be an integer vector is **NP**-complete.

PRIMES/COMPOSITE

COMPOSITE
Instance: Positive integer K
Question: Is K composite?

- PRIMES is the problem of deciding whether K is prime. For 25 years, this problem has been known to be in **NP** $\cap$ **coNP**.
  (we will discuss about **coNP** in lecture 8.)
- In 2002, Manindra Agrawal and two of his undergraduate students prove that PRIMES/COMPOSITE is in **P**.
- See Granville: tinyurl.com/695dj7
  It is easy to determine whether a given integer is prime.

Graph Isomorphism

Graph Isomorphism (GI)
Instance: Two undirected graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$
Question: Are G and H *isomorphic*, i.e., is there a *bijection* $f : V_G \to V_H$ such that $(u, v) \in E_G$ iff $(f(u), f(g)) \in E_H$?

- The status of this problem remains largely unknown.
- We know that if GI is **NP**-complete then this would have a lot of surprising consequences (such as collapse of the polynomial hierarchy).
- GI is currently the main candidate for **NP**-intermediate.
- Note that a similar problem Subgraph Isomorphism (where f is required only to be injective) is **NP**-complete.